

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 13.

**«Проектирование и реализация сервисного слоя с использованием
паттернов Repository, Service Layer, DTO и внедрения
зависимостей».**

Цель: развить навыки проектирования и реализации сервисного слоя с использованием архитектурных паттернов, обеспечивающих слабую связанность, тестируемость и расширяемость.

Выполнил: _____,

Студент 24-ИС.

2026г.

Введение

Общее условие (для всех вариантов)

Разработать сервисный слой для системы управления гостиницей с поддержкой:

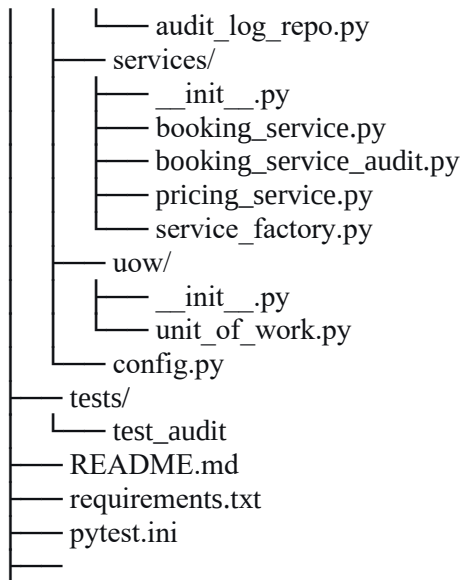
- 1. Управления отелями и номерами (CRUD).
- 2. Бронирования номеров с проверкой доступности.
- 3. Отмены бронирования с возвратом номера в пул.
- 4. Расчет стоимости с учетом сезонных коэффициентов (Strategy Pattern).
- 5. Поиска доступных номеров по датам и фильтрам.

Специфические требования к сервисному слою

Вариант	Бизнес-правило	Дополнительный паттерн
5	История изменений (Audit Log): логировать все действия с бронированиями (кто, когда, что изменил)	Decorator для логирования

Структура

```
booking_system/  
├── src/  
│   ├── __init__.py  
│   ├── domain/  
│   │   ├── __init__.py  
│   │   ├── models.py  
│   │   └── exceptions.py  
│   ├── dto/  
│   │   ├── __init__.py  
│   │   ├── booking_dto.py  
│   │   └── audit_log_dto.py  
│   └── repositories/  
│       ├── __init__.py  
│       ├── base.py  
│       ├── hotel_repo.py  
│       ├── room_repo.py  
│       └── booking_repo.py
```



Код

src

domain

```
# src/domain/models.py
from dataclasses import dataclass, field
from datetime import date, datetime
from enum import Enum
from typing import Optional

class BookingStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    CHECKED_OUT = "checked_out"
    CANCELLED = "cancelled"

class AuditActionType(Enum):
    CREATE = "create"
    UPDATE = "update"
    DELETE = "delete"
    CANCEL = "cancel"
    CONFIRM = "confirm"
    CHECK_IN = "check_in"
    CHECK_OUT = "check_out"
    STATUS_CHANGE = "status_change"

@dataclass
class Hotel:
    id: Optional[int]
    name: str
    address: str
    phone: str
```

```

        rating: float = 0.0
        created_at: datetime = field(default_factory=datetime.now)

@dataclass
class Room:
    id: Optional[int]
    hotel_id: int
    number: str
    capacity: int
    price_per_night: float
    is_active: bool = True
    room_type: str = "standard" # standard, deluxe, suite

@dataclass
class Booking:
    id: Optional[int]
    room_id: int
    guest_name: str
    guest_email: str
    check_in: date
    check_out: date
    total_price: float
    status: BookingStatus = BookingStatus.PENDING
    created_at: datetime = field(default_factory=datetime.now)
    cancelled_at: Optional[datetime] = None

@dataclass
class AuditLog:
    id: Optional[int]
    booking_id: int
    action_type: AuditActionType
    user_id: Optional[int]
    user_role: Optional[str]
    old_data: Optional[dict] = None
    new_data: Optional[dict] = None
    changed_fields: Optional[list] = None
    ip_address: Optional[str] = None
    user_agent: Optional[str] = None
    created_at: datetime = field(default_factory=datetime.now)
    description: Optional[str] = None

```

Приложение 1. models

```

# src/domain/exceptions.py
class DomainError(Exception):
    """Базовое исключение домена"""
    def __init__(self, message: str, details: dict = None):
        self.message = message
        self.details = details or {}
        super().__init__(message)

class RoomNotFoundError(DomainError):

```

```

    pass

class RoomNotAvailableError(DomainError):
    pass

class BookingNotFoundError(DomainError):
    pass

class BookingConflictError(DomainError):
    pass

class InvalidDatesError(DomainError):
    pass

class HotelNotFoundError(DomainError):
    pass

class AuditLogError(DomainError):
    """Ошибка при работе с аудит-логами"""
    pass

```

Приложение 2. exceptions

dto

```

# src/dto/audit_log_dto.py
from pydantic import BaseModel
from datetime import datetime
from typing import Optional, List
from ..domain.models import AuditActionType

class AuditLogCreatedDTO(BaseModel):
    """DTO для создания аудит-лога"""
    booking_id: int
    action_type: AuditActionType
    user_id: Optional[int] = None
    user_role: Optional[str] = None
    old_data: Optional[dict] = None
    new_data: Optional[dict] = None
    changed_fields: Optional[List[str]] = None
    ip_address: Optional[str] = None
    user_agent: Optional[str] = None
    description: Optional[str] = None

class AuditLogResponseDTO(BaseModel):
    """DTO для ответа с аудит-логом"""
    id: int
    booking_id: int
    action_type: str
    user_id: Optional[int]

```

```

user_role: Optional[str]
old_data: Optional[dict]
new_data: Optional[dict]
changed_fields: Optional[List[str]]
ip_address: Optional[str]
user_agent: Optional[str]
created_at: datetime
description: Optional[str]

class AuditLogFilterDTO(BaseModel):
    """DTO для фильтрации аудит-логов"""
    booking_id: Optional[int] = None
    action_type: Optional[AuditActionType] = None
    user_id: Optional[int] = None
    date_from: Optional[datetime] = None
    date_to: Optional[datetime] = None
    limit: int = 100
    offset: int = 0

```

Приложение 3. audit_log_dto

```

# src/dto/booking_dto.py
from pydantic import BaseModel, field_validator
from datetime import date, datetime
from typing import Optional

class BookingCreateDTO(BaseModel):
    room_id: int
    guest_name: str
    guest_email: str
    check_in: date
    check_out: date

    @field_validator('check_out')
    @classmethod
    def validate_dates(cls, v: date, info) -> date:
        """Валидация дат бронирования"""
        values = info.data
        if 'check_in' in values:
            check_in = values['check_in']
            if v <= check_in:
                raise ValueError('Дата выезда должна быть позже даты заезда')
            if (v - check_in).days > 30:
                raise ValueError('Бронирование не может превышать 30 дней')
        return v

class BookingResponseDTO(BaseModel):
    id: int
    room_id: int

```

```

    guest_name: str
    check_in: date
    check_out: date
    total_price: float
    status: str
    created_at: datetime

class BookingUpdateDTO(BaseModel):
    guest_name: Optional[str] = None
    guest_email: Optional[str] = None

```

Приложение 4. Booking_dto

repositories

```

# src/repositories/audit_log_repo.py
from typing import List, Optional, Dict, Any
from datetime import datetime
from src.domain.models import AuditLog
from src.repositories.base import BaseRepository

class AuditLogRepository(BaseRepository[AuditLog]):
    """Репозиторий для работы с аудит-логами (In-Memory)"""

    def __init__(self):
        self._storage: Dict[int, AuditLog] = {}
        self._next_id = 1

    def get_by_id(self, id: int) -> Optional[AuditLog]:
        return self._storage.get(id)

    def get_all(self, **filters) -> List[AuditLog]:
        result = list(self._storage.values())

        if 'booking_id' in filters:
            result = [log for log in result if log.booking_id ==
filters['booking_id']]

        if 'action_type' in filters:
            result = [log for log in result if log.action_type ==
filters['action_type']]

        if 'user_id' in filters:
            result = [log for log in result if log.user_id == filters['user_id']]

        if 'date_from' in filters:
            result = [log for log in result if log.created_at >=
filters['date_from']]

        if 'date_to' in filters:

```

```

        result = [log for log in result if log.created_at <=
filters['date_to']]

    # Сортировка по времени создания (новые сверху)
    result.sort(key=lambda x: x.created_at, reverse=True)

    return result

def add(self, log: AuditLog) -> AuditLog:
    log.id = self._next_id
    self._storage[log.id] = log
    self._next_id += 1
    return log

def update(self, log: AuditLog) -> AuditLog:
    if log.id not in self._storage:
        raise ValueError(f"AuditLog with id {log.id} not found")
    self._storage[log.id] = log
    return log

def delete(self, id: int) -> bool:
    if id in self._storage:
        del self._storage[id]
        return True
    return False

def get_by_booking(self, booking_id: int, limit: int = 100, offset: int = 0)
-> List[AuditLog]:
    """Получить все аудит-логи для конкретного бронирования"""
    logs = [log for log in self._storage.values() if log.booking_id ==
booking_id]
    logs.sort(key=lambda x: x.created_at, reverse=True)
    return logs[offset:offset + limit]

```

Приложение 5. audit_log_repo

```

from abc import ABC, abstractmethod
from typing import List, Optional, TypeVar, Generic

T = TypeVar('T')

class BaseRepository(ABC, Generic[T]):
    @abstractmethod
    def get_by_id(self, id: int) -> Optional[T]:
        pass

    @abstractmethod
    def get_all(self, **filters) -> List[T]:
        pass

    @abstractmethod
    def add(self, entity: T) -> T:

```



```

        pass

    @abstractmethod
    def update(self, entity: T) -> T:
        pass

    @abstractmethod
    def delete(self, id: int) -> bool:
        pass

```

Приложение 6. base

```

# src/repositories/booking_repo.py
from typing import List, Optional
from datetime import date
from src.domain.models import Booking
from src.repositories.base import BaseRepository

class BookingRepository(BaseRepository[Booking]):
    def __init__(self):
        self._storage: dict[int, Booking] = {}
        self._next_id = 1

    def get_by_id(self, id: int) -> Optional[Booking]:
        return self._storage.get(id)

    def get_all(self, **filters) -> List[Booking]:
        result = list(self._storage.values())
        # Применяем фильтры
        if 'room_id' in filters:
            result = [b for b in result if b.room_id == filters['room_id']]
        if 'status' in filters:
            result = [b for b in result if b.status == filters['status']]
        return result

    def get_by_room_and_dates(
        self,
        room_id: int,
        check_in: date,
        check_out: date
    ) -> List[Booking]:
        """Найти бронирования для номера в указанном диапазоне"""
        result = []
        for booking in self._storage.values():
            if booking.room_id != room_id:
                continue
            if booking.status == BookingStatus.CANCELLED:
                continue
            # Пересечение интервалов
            if booking.check_in < check_out and booking.check_out > check_in:

```

```

        result.append(booking)
    return result

def add(self, booking: Booking) -> Booking:
    booking.id = self._next_id
    self._storage[booking.id] = booking
    self._next_id += 1
    return booking

def update(self, booking: Booking) -> Booking:
    if booking.id not in self._storage:
        raise ValueError(f"Booking with id {booking.id} not found")
    self._storage[booking.id] = booking
    return booking

def delete(self, id: int) -> bool:
    if id in self._storage:
        del self._storage[id]
        return True
    return False

```

Приложение 7. booking_repo

```

from typing import List, Optional
from datetime import date
from src.domain.models import Hotel
from src.repositories.base import BaseRepository

class HotelRepository(BaseRepository[Hotel]):
    def __init__(self):
        self._storage: dict[int, Hotel] = {}
        self._next_id = 1

    def get_by_id(self, id: int) -> Optional[Hotel]:
        return self._storage.get(id)

    def get_all(self, **filters) -> List[Hotel]:
        result = list(self._storage.values())
        # Применяем фильтры
        if 'id' in filters:
            result = [b for b in result if b.room_id == filters['id']]
        return result

    def add(self, hotel: Hotel) -> Hotel:
        hotel.id = self._next_id
        self._storage[hotel.id] = hotel
        self._next_id += 1
        return hotel

    def update(self, hotel: Hotel) -> Hotel:
        if hotel.id not in self._storage:

```

```

        raise ValueError(f"Hotel with id {hotel.id} not found")
    self._storage[hotel.id] = hotel
    return hotel

def delete(self, id: int) -> bool:
    if id in self._storage:
        del self._storage[id]
        return True
    return False

```

Приложение 8. hotel_repo

```

# src/repositories/room_repo.py
from typing import List, Optional
from ..domain.models import Room
from .base import BaseRepository

class RoomRepository(BaseRepository[Room]):
    def __init__(self):
        self._storage: dict[int, Room] = {}
        self._next_id = 1

    def get_by_id(self, id: int) -> Optional[Room]:
        return self._storage.get(id)

    def get_all(self, **filters) -> List[Room]:
        result = list(self._storage.values())

        if 'hotel_id' in filters:
            result = [r for r in result if r.hotel_id == filters['hotel_id']]
        if 'is_active' in filters:
            result = [r for r in result if r.is_active == filters['is_active']]

        return result

    def add(self, room: Room) -> Room:
        room.id = self._next_id # Исправлено: room.id, а не Room.id
        self._storage[room.id] = room
        self._next_id += 1
        return room

    def update(self, room: Room) -> Room:
        if room.id not in self._storage:
            raise ValueError(f"Room with id {room.id} not found")
        self._storage[room.id] = room
        return room

    def delete(self, id: int) -> bool:
        if id in self._storage:
            del self._storage[id]

```

```

        return True
    return False

    def get_by_hotel(self, hotel_id: int, active_only: bool = True) ->
List[Room]:
    """Получить все номера отеля"""
    result = [r for r in self._storage.values() if r.hotel_id == hotel_id]
    if active_only:
        result = [r for r in result if r.is_active]
    return result

```

Приложение 9. room_repo

services

```

# src/services/booking_service.py
from datetime import date, datetime
from typing import List, Optional, Dict, Any
from ..domain.models import Booking, BookingStatus
from ..domain.exceptions import (
    RoomNotFoundError, RoomNotAvailableError,
    BookingConflictError, BookingNotFoundError, InvalidDatesError, DomainError
)
from ..dto.booking_dto import BookingCreatedDTO, BookingResponseDTO,
BookingUpdatedDTO
from ..uow.unit_of_work import UnitOfWork
from .pricing_service import PricingService

class BookingService:
    """Базовый сервис для управления бронированиями (без аудита)"""

    def __init__(self, uow: UnitOfWork, pricing_service: PricingService):
        self.uow = uow
        self.pricing_service = pricing_service
        self.booking_repo = uow.bookings
        self.room_repo = uow.rooms
        # Контекст выполнения для аудита
        self._context = {
            'user_id': None,
            'user_role': 'system',
            'ip_address': None,
            'user_agent': None
        }

    def set_context(self, user_id: Optional[int] = None,
                    user_role: str = 'system',
                    ip_address: Optional[str] = None,
                    user_agent: Optional[str] = None):
        """Установить контекст выполнения"""
        self._context.update({
            'user_id': user_id,

```

```

        'user_role': user_role,
        'ip_address': ip_address,
        'user_agent': user_agent
    })

def get_context(self) -> dict:
    """Получить контекст выполнения"""
    return self._context.copy()

def create(self, dto: BookingCreateDTO) -> BookingResponseDTO:
    """Создать новое бронирование"""
    # Проверяем существование номера
    room = self.room_repo.get_by_id(dto.room_id)
    if not room:
        raise RoomNotFoundError(f"Номер {dto.room_id} не найден")
    if not room.is_active:
        raise RoomNotFoundError(f"Номер {dto.room_id} не активен")

    # Проверяем пересечения бронирований
    existing = self.booking_repo.get_by_room_and_dates(
        dto.room_id, dto.check_in, dto.check_out
    )
    if existing:
        raise BookingConflictError(
            f"Номер {dto.room_id} уже забронирован на эти даты",
            details={"conflicting_bookings": [b.id for b in existing]}
        )

    # Рассчитываем стоимость
    total_price = self.pricing_service.calculate_price(
        room, dto.check_in, dto.check_out
    )

    # Создаем бронирование
    booking = Booking(
        id=None,
        room_id=dto.room_id,
        guest_name=dto.guest_name,
        guest_email=dto.guest_email,
        check_in=dto.check_in,
        check_out=dto.check_out,
        total_price=total_price,
        status=BookingStatus.PENDING
    )

    # Сохраняем
    saved = self.booking_repo.add(booking)
    self.uow.commit()

    return BookingResponseDTO(
        id=saved.id,

```

```

        room_id=saved.room_id,
        guest_name=saved.guest_name,
        check_in=saved.check_in,
        check_out=saved.check_out,
        total_price=saved.total_price,
        status=saved.status.value,
        created_at=saved.created_at
    )

def cancel(self, booking_id: int) -> bool:
    """Отменить бронирование"""
    booking = self.booking_repo.get_by_id(booking_id)
    if not booking:
        raise BookingNotFoundError(f"Бронирование {booking_id} не найдено")

    if booking.status in (BookingStatus.CHECKED_IN,
BookingStatus.CHECKED_OUT):
        raise DomainError(
            f"Нельзя отменить бронирование в статусе {booking.status.value}"
        )

    booking.status = BookingStatus.CANCELLED
    booking.cancelled_at = datetime.now()
    booking.updated_at = datetime.now()

    self.booking_repo.update(booking)
    self.uow.commit()

    return True

def confirm(self, booking_id: int) -> None:
    """Подтвердить бронирование"""
    booking = self.booking_repo.get_by_id(booking_id)
    if not booking:
        raise BookingNotFoundError(f"Бронирование {booking_id} не найдено")

    if booking.status != BookingStatus.PENDING:
        raise DomainError(
            f"Бронирование в статусе {booking.status.value} нельзя
подтвердить"
        )

    booking.status = BookingStatus.CONFIRMED
    booking.updated_at = datetime.now()

    self.booking_repo.update(booking)
    self.uow.commit()

def update(self, booking_id: int, dto: BookingUpdateDTO) ->
BookingResponseDTO:
    """Обновить данные бронирования"""

```

```

        booking = self.booking_repo.get_by_id(booking_id)
        if not booking:
            raise BookingNotFoundError(f"Бронирование {booking_id} не найдено")

        if booking.status in (BookingStatus.CANCELLED,
BookingStatus.CHECKED_OUT):
            raise DomainError(f"Нельзя изменить бронирование в статусе
{booking.status.value}")

        # Обновляем поля
        if dto.guest_name is not None:
            booking.guest_name = dto.guest_name
        if dto.guest_email is not None:
            booking.guest_email = dto.guest_email
        booking.updated_at = datetime.now()

        self.booking_repo.update(booking)
        self.uow.commit()

        return BookingResponseDTO(
            id=booking.id,
            room_id=booking.room_id,
            guest_name=booking.guest_name,
            check_in=booking.check_in,
            check_out=booking.check_out,
            total_price=booking.total_price,
            status=booking.status.value,
            created_at=booking.created_at
        )

    def get_available_rooms(
        self,
        hotel_id: int,
        check_in: date,
        check_out: date,
        capacity: Optional[int] = None
    ) -> List[dict]:
        """Получить доступные номера в отеле на указанные даты"""
        # 1. Получаем все номера отеля
        rooms = self.room_repo.get_by_hotel(hotel_id, active_only=True)

        # 2. Фильтруем по вместимости
        if capacity:
            rooms = [r for r in rooms if r.capacity >= capacity]

        # 3. Для каждого номера проверяем доступность
        available = []
        for room in rooms:
            existing = self.booking_repo.get_by_room_and_dates(
                room.id, check_in, check_out
            )

```

```

        if not existing:
            available.append({
                'room_id': room.id,
                'number': room.number,
                'capacity': room.capacity,
                'price_per_night': room.price_per_night
            })

    return available

def get_by_id(self, booking_id: int) -> Optional[Booking]:
    """Получить бронирование по ID"""
    return self.booking_repo.get_by_id(booking_id)

def get_all(self, **filters) -> List[Booking]:
    """Получить все бронирования с фильтрами"""
    return self.booking_repo.get_all(**filters)

```

Приложение 10. booking_service

```

# src/services/booking_service_audit.py
from functools import wraps
from typing import Callable, Any, Optional
from datetime import datetime
import inspect

from ..domain.models import AuditLog, AuditActionType, Booking
from ..domain.exceptions import DomainError
from ..dto.audit_log_dto import AuditLogCreatedDTO
from .booking_service import BookingService
from ..uow.unit_of_work import UnitOfWork

class AuditDecorator:
    """
    Декоратор для логирования действий с бронированиями.
    Использует паттерн Decorator для добавления функциональности аудита.
    """

    def __init__(self, service: BookingService):
        """
        Инициализация декоратора.

        Args:
            service: Базовый сервис бронирований
        """
        self._service = service
        self.uow = service.uow

    def set_context(self, user_id: Optional[int] = None,
                   user_role: str = 'system',
                   ip_address: Optional[str] = None,

```



```

        user_agent: Optional[str] = None):
    """Установить контекст выполнения"""
    self._service.set_context(user_id, user_role, ip_address, user_agent)

def __getattr__(self, name):
    """
    Проксирование всех методов к базовому сервису.
    Декорируем только нужные методы.
    """
    attr = getattr(self._service, name)
    if callable(attr) and name in ['create', 'cancel', 'confirm', 'update']:
        return self._decorate_method(name, attr)
    return attr

def _decorate_method(self, method_name: str, method: Callable) -> Callable:
    """
    Декорирует метод для добавления аудита.
    """
    @wraps(method)
    def wrapper(*args, **kwargs):
        # Получаем контекст из сервиса
        context = self._service.get_context()
        user_id = context.get('user_id')
        user_role = context.get('user_role', 'system')
        ip_address = context.get('ip_address')
        user_agent = context.get('user_agent')

        # Определяем действие и получаем данные до выполнения
        action_type, booking_id, old_data = self._prepare_audit_data(
            method_name, args, kwargs
        )

        try:
            # Выполняем основной метод
            result = method(*args, **kwargs)

            # Получаем данные после выполнения
            new_data = self._get_booking_data(booking_id) if booking_id else
None

            # Для создания бронирования - получаем ID из результата
            if action_type == AuditActionType.CREATE and result:
                booking_id = result.id
                new_data = self._get_booking_data(booking_id)

            # Создаем аудит-лог
            self._create_audit_log(
                booking_id=booking_id if booking_id else 0,
                action_type=action_type,
                user_id=user_id,
                user_role=user_role,

```

```

        old_data=old_data,
        new_data=new_data,
        ip_address=ip_address,
        user_agent=user_agent,
        description=f"{action_type.value} booking {booking_id if
booking_id else 'new'}"
    )

    return result

except DomainError as e:
    # Логируем ошибку в аудит
    self._create_audit_log(
        booking_id=booking_id if booking_id else 0,
        action_type=action_type,
        user_id=user_id,
        user_role=user_role,
        old_data=old_data,
        new_data=None,
        ip_address=ip_address,
        user_agent=user_agent,
        description=f"ERROR: {str(e)}"
    )
    raise

return wrapper

def _prepare_audit_data(self, method_name: str, args: tuple, kwargs: dict) ->
tuple:
    """
    Подготавливает данные для аудита.
    Возвращает (action_type, booking_id, old_data)
    """
    if method_name == 'create':
        return AuditActionType.CREATE, None, None

    elif method_name == 'cancel':
        booking_id = args[0] if args else kwargs.get('booking_id')
        old_data = self._get_booking_data(booking_id) if booking_id else None
        return AuditActionType.CANCEL, booking_id, old_data

    elif method_name == 'confirm':
        booking_id = args[0] if args else kwargs.get('booking_id')
        old_data = self._get_booking_data(booking_id) if booking_id else None
        return AuditActionType.CONFIRM, booking_id, old_data

    elif method_name == 'update':
        booking_id = args[0] if args else kwargs.get('booking_id')
        old_data = self._get_booking_data(booking_id) if booking_id else None
        return AuditActionType.UPDATE, booking_id, old_data

```

```

        return AuditActionType.UPDATE, None, None

    def _get_booking_data(self, booking_id: int) -> Optional[dict]:
        """Получает данные бронирования для аудита"""
        booking = self.uow.bookings.get_by_id(booking_id)
        if not booking:
            return None

        return {
            'id': booking.id,
            'room_id': booking.room_id,
            'guest_name': booking.guest_name,
            'guest_email': booking.guest_email,
            'check_in': booking.check_in.isoformat() if booking.check_in else
None,
            'check_out': booking.check_out.isoformat() if booking.check_out else
None,
            'total_price': booking.total_price,
            'status': booking.status.value if booking.status else None,
            'created_at': booking.created_at.isoformat() if booking.created_at
else None,
        }

    def _get_changed_fields(self, old_data: Optional[dict], new_data:
Optional[dict]) -> list:
        """Определяет измененные поля"""
        if not old_data or not new_data:
            return []

        changed = []
        for key in old_data.keys():
            if key in new_data and old_data[key] != new_data[key]:
                changed.append(key)
        return changed

    def _create_audit_log(
        self,
        booking_id: int,
        action_type: AuditActionType,
        user_id: Optional[int],
        user_role: str,
        old_data: Optional[dict],
        new_data: Optional[dict],
        ip_address: Optional[str],
        user_agent: Optional[str],
        description: str
    ):
        """
        Создает запись в аудит-логе.
        """
        # Определяем измененные поля

```

```

changed_fields = self._get_changed_fields(old_data, new_data)

# Создаем сущность AuditLog
audit_log = AuditLog(
    id=None,
    booking_id=booking_id,
    action_type=action_type,
    user_id=user_id,
    user_role=user_role,
    old_data=old_data,
    new_data=new_data,
    changed_fields=changed_fields,
    ip_address=ip_address,
    user_agent=user_agent,
    created_at=datetime.now(),
    description=description
)

# Сохраняем аудит-лог
self.uow.audit_logs.add(audit_log)
self.uow.commit()

```

Приложение 11. Booking_service_audit

```

# src/services/pricing_service.py
from datetime import date
from typing import Optional
from ..domain.models import Room
from ..domain.exceptions import InvalidDatesError

class PricingService:
    """Сервис расчета стоимости с использованием стратегий"""

    def __init__(self, seasonal_coefficients: Optional[dict] = None):
        self.seasonal_coefficients = seasonal_coefficients or {
            # месяц: коэффициент
            6: 1.2, # июнь
            7: 1.5, # июль
            8: 1.5, # август
            12: 1.3, # декабрь (новый год)
            1: 1.1, # январь
        }

    def calculate_price(
        self,
        room: Room,
        check_in: date,
        check_out: date
    ) -> float:
        """Рассчитать стоимость бронирования"""
        nights = (check_out - check_in).days

```

```

    if nights <= 0:
        raise InvalidDatesError("Количество ночей должно быть больше 0")

    # Базовая цена
    total = 0.0
    current_date = check_in
    while current_date < check_out:
        # Применяем сезонный коэффициент
        month = current_date.month
        coefficient = self.seasonal_coefficients.get(month, 1.0)
        total += room.price_per_night * coefficient

        # Переходим к следующему месяцу
        if current_date.month == 12:
            current_date = date(current_date.year + 1, 1, 1)
        else:
            current_date = date(current_date.year, current_date.month + 1, 1)

    # Дополнительная скидка за длительное бронирование
    if nights >= 7:
        total *= 0.95 # 5% скидка
    if nights >= 14:
        total *= 0.9 # дополнительная скидка (всего 14.5%)

    return round(total, 2)

```

Приложение 11. pricing_service

```

# src/services/service_factory.py
from src.services.booking_service import BookingService
from src.services.booking_service_audit import AuditDecorator
from src.services.pricing_service import PricingService
from src.uow.unit_of_work import UnitOfWork

class ServiceFactory:
    """Фабрика для создания сервисов с или без аудита"""

    @staticmethod
    def create_booking_service(
        uow: UnitOfWork,
        pricing_service: PricingService,
        enable_audit: bool = True
    ) -> BookingService:
        """
        Создает сервис бронирований.

        Args:
            uow: Unit of Work
            pricing_service: Сервис ценообразования
            enable_audit: Включить ли аудит

        Returns:

```

```

        BookingService: Сервис бронирований (с аудитом или без)
    """
    base_service = BookingService(uow, pricing_service)

    if enable_audit:
        # Оборачиваем декоратором для добавления аудита
        return AuditDecorator(base_service)

    return base_service

```

Приложение 12. service_factory

uow

```

# src/uow/unit_of_work.py
from contextlib import contextmanager
from typing import Type
from src.repositories.booking_repo import BookingRepository
from src.repositories.hotel_repo import HotelRepository
from src.repositories.room_repo import RoomRepository
from src.repositories.audit_log_repo import AuditLogRepository

class UnitOfWork:
    def __init__(self):
        self._hotel_repo = HotelRepository()
        self._room_repo = RoomRepository()
        self._booking_repo = BookingRepository()
        self._audit_log_repo = AuditLogRepository()
        self._committed = False

    @property
    def hotels(self) -> HotelRepository:
        return self._hotel_repo

    @property
    def rooms(self) -> RoomRepository:
        return self._room_repo

    @property
    def bookings(self) -> BookingRepository:
        return self._booking_repo

    @property
    def audit_logs(self) -> AuditLogRepository:
        return self._audit_log_repo

    def commit(self) -> None:
        """Фиксация транзакции"""
        self._committed = True

    def rollback(self) -> None:
        """Откат транзакции"""

```

```

        self._committed = False

    @contextmanager
    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            self.rollback()
        elif not self._committed:
            self.commit()

```

Приложение 13. Unit_of_work

tests

```

# day13/test_audit.py
import sys
import os

# Добавляем текущую директорию в PYTHONPATH
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from src.domain.models import Hotel, Room
from src.services.booking_service import BookingService
from src.services.booking_service_audit import AuditDecorator
from src.services.pricing_service import PricingService
from src.uow.unit_of_work import UnitOfWork
from src.dto.booking_dto import BookingCreatedDTO
from datetime import date

def test_audit():
    print("=" * 50)
    print("Тестирование аудит-логов")
    print("=" * 50)

    # Создаем UoW
    uow = UnitOfWork()

    # Создаем тестовый отель и номер
    print("\n1. Создание тестовых данных...")
    hotel = Hotel(id=None, name="Test Hotel", address="Test Address",
phone="+123456789")
    saved_hotel = uow.hotels.add(hotel)
    print(f"    ✅ Создан отель ID: {saved_hotel.id}")

    room = Room(id=None, hotel_id=saved_hotel.id, number="101", capacity=2,
price_per_night=100.0)
    saved_room = uow.rooms.add(room)
    print(f"    ✅ Создан номер ID: {saved_room.id}")

    # Создаем сервис с аудитом

```

```

pricing_service = PricingService()
booking_service = BookingService(uow, pricing_service)
audit_service = AuditDecorator(booking_service)

# Устанавливаем контекст выполнения
audit_service.set_context(
    user_id=1,
    user_role="admin",
    ip_address="192.168.1.100",
    user_agent="Mozilla/5.0"
)

# Создаем бронирование
dto = BookingCreateDTO(
    room_id=saved_room.id,
    guest_name="John Doe",
    guest_email="john@example.com",
    check_in=date(2026, 6, 15),
    check_out=date(2026, 6, 20)
)

print("\n2. Создание бронирования...")
try:
    result = audit_service.create(dto)
    print(f"    ✓ Создано бронирование ID: {result.id}")
    print(f"    💰 Стоимость: {result.total_price} руб.")
except Exception as e:
    print(f"    ✗ Ошибка: {e}")
    return

print("\n3. Проверка аудит-логов...")
logs = uow.audit_logs.get_all()
print(f"    📋 Всего аудит-логов: {len(logs)}")

for log in logs:
    print(f"    - {log.action_type.value}: бронирование {log.booking_id}")
    print(f"    Пользователь: {log.user_role} (ID: {log.user_id})")

print("\n4. Отмена бронирования...")
try:
    audit_service.cancel(result.id)
    print(f"    ✓ Бронирование {result.id} отменено")
except Exception as e:
    print(f"    ✗ Ошибка: {e}")

print("\n5. Проверка аудит-логов после отмены...")
logs = uow.audit_logs.get_all()
print(f"    📋 Всего аудит-логов: {len(logs)}")

for log in logs:
    print(f"    - {log.action_type.value}: бронирование {log.booking_id}")

```



```

        if log.old_data:
            old_status = log.old_data.get('status', 'N/A')
            print(f"    Старый статус: {old_status}")
        if log.new_data:
            new_status = log.new_data.get('status', 'N/A')
            print(f"    Новый статус: {new_status}")
        if log.changed_fields:
            print(f"    Измененные поля: {'', '.join(log.changed_fields)}")

print("\n6. Получение аудит-логов по бронированию...")
booking_logs = uow.audit_logs.get_by_booking(result.id)
print(f"    📄 Найдено {len(booking_logs)} логов для бронирования
{result.id}")

print("\n7. Детальная информация об аудит-логах:")
for log in booking_logs:
    print(f"    📄 Лог #{log.id}:")
    print(f"        Действие: {log.action_type.value}")
    print(f"        Время: {log.created_at}")
    print(f"        Описание: {log.description}")
    if log.old_data:
        print(f"            Было: {log.old_data.get('status', 'N/A')}")
    if log.new_data:
        print(f"            Стало: {log.new_data.get('status', 'N/A')}")

print("\n" + "=" * 50)
print("✅ Тест завершен успешно!")

if __name__ == "__main__":
    test_audit()

```

Приложение 14. test_audit

Тест

```

===== 1 passed in 0.70s =====
Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day13>python -m pytest tests/test_audit.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student240\AppData\Local\Programs\Python\Python
13\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day13
configfile: pytest.ini
plugins: hypothesis-6.155.3
collected 1 item

tests/test_audit.py::test_audit PASSED [100%]

===== 1 passed in 0.63s =====
Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day13>

```

Скр.1. Тест в cmd

Вопросы

1. Инкапсуляция бизнес-логики, отделение её от контроллеров и репозиториев для обеспечения переиспользования, тестируемости и поддерживаемости кода.
2. DTO (Data Transfer Object) — это объект для передачи данных между слоями (содержит только данные, без логики), а Domain Model — это бизнес-сущность с поведением и бизнес-правилами.
3. Unit of Work — это паттерн, который управляет транзакциями и гарантирует, что все изменения в нескольких репозиториях будут сохранены атомарно (все вместе или ничего) через commit/rollback.
4. Repository отвечает за доступ к данным (CRUD, фильтрация), а Service содержит бизнес-логику и использует репозитории для выполнения операций.
5. Внедрение зависимостей решает проблему жесткой связанности, позволяя легко заменять зависимости, упрощая тестирование и делая код более гибким и поддерживаемым.
6. Потому что это нарушает принцип единственной ответственности, приводит к дублированию кода, усложняет тестирование и делает невозможным переиспользование бизнес-логики в других интерфейсах (CLI, очереди).
7. Кастомные бизнес-исключения (DomainError), такие как RoomNotFoundError, BookingConflictError, InvalidDatesError, которые отражают конкретные бизнес-ситуации.
8. Используя моки (Mock) репозиториев в unit-тестах — подменяя реальные репозитории на имитации, которые возвращают заранее заданные данные и проверяют вызовы методов.

Вывод

ходе выполнения практической работы была успешно разработана система управления бронированиями с реализацией сервисного слоя на основе архитектурных паттернов. Проект продемонстрировал эффективность применения современных подходов к разработке серверных приложений на Python.